

3D Ray-casting Game with Cub3D 🧠

13 min read · Jul 26, 2025



Abdilah CH

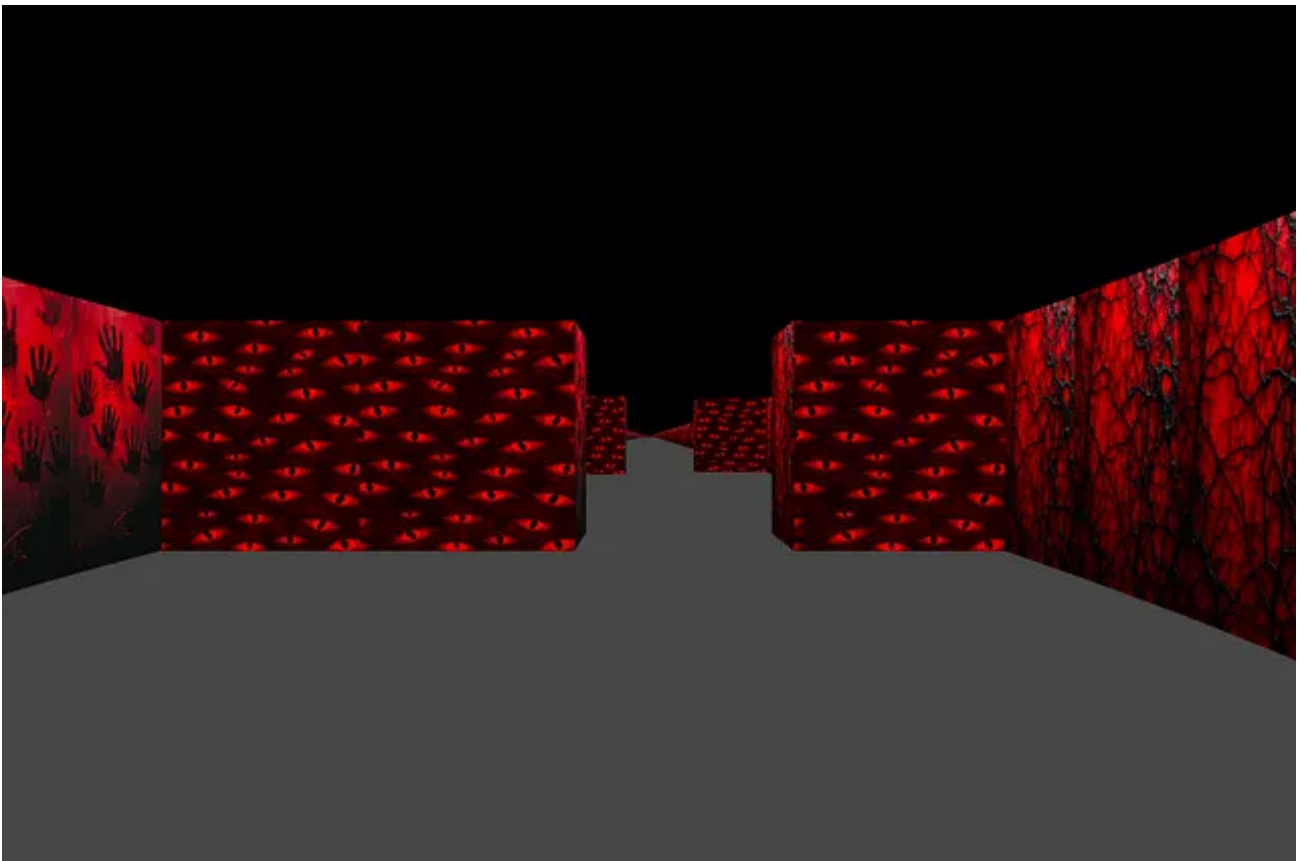
Follow



Listen



Share



🧩 Introduction

Cub3D is a raycasting project where you simulate a 3D view using a 2D grid map, similar to how early 3D games like *Wolfenstein 3D* worked.

The objective is to create a first-person environment using only:

- A 2D map
- Raycasting to determine visibility
- A graphical library (**MiniLibX**) for rendering

💡 This project develops your understanding of:

- Game loops and rendering
- Vectors, distances, and projections
- DDA and raycasting logic
- Input handling and player interaction

. . .

Parsing and Validating

◆ PART 1: Texture Paths (NO, SO, WE, EA)

Goal:

Tell the program which texture image to apply on each side of the wall.

Format:

Each texture line starts with a 2-letter identifier, followed by a relative or absolute path to a **.xpm** texture file:

```
NO ./textures/north.xpm
SO ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm
```

Rules:

- . You must have exactly one of each: NO, SO, WE, EA.
- . The file path must point to a readable .xpm file.
- . No duplicate identifiers (e.g., two NO lines).
- . No invalid identifiers (e.g., NX, ZZ, or missing one).

What happens if:

. A path is invalid or the file doesn't exist? → Parsing error, the program must exit.

. You forget one of them? → The rendering system won't work correctly.

. . .

◆ PART 2: Colors for Floor (F) and Ceiling

🎯 Goal:

Set the background color used to fill floor and ceiling.

📄 Format:

A single line with a letter and comma-separated RGB values:

```
F 220,100,0  
C 255,255,255
```

✅ Rules:

Only F (floor) and C (ceiling) are allowed.
RGB values must be:
. Exactly three numbers.
. Each number $\in [0, 255]$.
. No extra commas or missing values.
Must be only one F and one C.

⚠ Notes:

. Color is often stored as a single 24-bit integer:

$\text{color} = (\text{R} \ll 16) \mid (\text{G} \ll 8) \mid \text{B}$

. If any value is outside the valid range → error.

. . .

◆ PART 3: The Map

1	1	1	1	1	1	1
1	0	0	0	0	0	1
1	0	0	0	0	0	1
1	1	1	1	1	1	1

Goal:

Describe the layout of the 2D world where the player moves.

1111111	1 ---> Wall (solid, not walkable)
1000001	0 ---> Empty space (walkable)
10N0001	N/S/E/W ---> Player's spawn direction
1111111	(space) ---> Outside or padding

Rules:

- . The map must be enclosed by walls (1). No holes.
- . Only one of the player's start character (N, S, E, W) is allowed.
- . No invalid characters.
- . Spaces should only exist outside the playable map, not inside.
- . Map lines can have variable lengths (be careful to normalize).

. . .

◆ PART 4: Map Extraction and Normalization

🎯 Goal:

Correctly identify where the map begins in the file and extract only the map content.

How?

- . Skip identifier lines (NO, F, etc.) first.
- . After an empty line or after identifiers stop, remaining lines are part of the map.

⚠️ Pitfalls:

- . Some maps might have leading empty lines or inconsistent spacing.
- . You must handle lines with different lengths (e.g., fill missing parts with spaces).

. . .

◆ PART 5: Map Validation

🎯 Goal:

Make sure the map is safe and playable.

✅ What to check:

Check	Why it matters
Surrounded by walls	Prevent the player from e
Only allowed characters	Garbage chars crash the p
One player start	Must not have two or more
No adjacent `0` to space or out-of-bounds	Avoid leaks to void.

Techniques:

. **Flood Fill:** Start from player position, simulate movement in 4 directions.

If it reaches a space or out-of-bounds → invalid map.

. **Edge Scanning:** First and last line must be all 1 or space. Same for leftmost/rightmost characters.

. . .

◆ PART 6: Player Initialization

🎯 Goal:

Set up the player's starting:

- . Position in the world.
- . Direction vector (where they look).
- . Camera plane (for rendering perspective).
- . Spawn letters meaning.

Letter	Direction	
--------	-----------	--

	-----	-----	
	`N`	Facing up (y-)	
	`S`	Facing down (y+)	
	`E`	Facing right (x+)	
	`W`	Facing left (x-)	

Player Data:

Data	Purpose
-----	-----
`x`, `y`	Initial coordinates in the map
`dir_x`, `dir_y`	Where the player is looking
`plane_x`, `plane_y`	Used for FOV in raycasting (camera width)

The camera plane is always perpendicular to the direction vector.s



Player Direction & Camera Plane in Ray-casting



Part 1: The Map and the Player

Before doing any 3D rendering, we start with a 2D top-down map — a grid of tiles where:

```
1 = wall
0 = empty space
N, S, E, W = player starting position & direction
```

We need to figure out:

👉 Where is the player looking?

👉 What can the player see?

To do that, we use vectors.

. . .

Part 2: The Player Direction Vector

The direction vector represents the direction the player is currently facing. This vector has two parts:

dirX: horizontal component
dirY: vertical component

Think of it as an arrow starting at the player's position and pointing forward.

Example directions:

Facing	dirX	dirY	Vector Arrow
North	0	-1	↑
South	0	+1	↓
East	+1	0	→
West	-1	0	←

Why a Vector?

Because it gives us precise direction — not just N/S/E/W, but any in-between angles too (when the player rotates).

So if a player is looking slightly right and up, the vector might be:

dirX = 0.7
dirY = -0.7



. . .

Part 3: The Camera Plane Vector

Now that we know where the player is looking, we need to define their field of view (FOV) — how wide they can see.

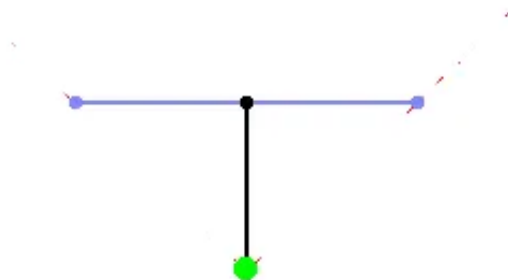
This is done with the camera plane vector.



What is the camera plane?

It's a vector perpendicular to the player's direction vector.
It determines the left and right edge of what the player sees.
It doesn't point "forward", but rather left/right across the screen.

Imagine it like this:



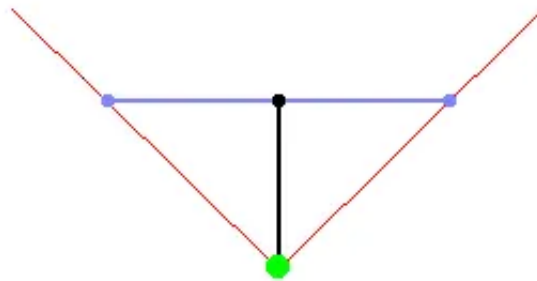
. . .

▲ Part 4: Viewing Cone (FOV)

When we combine:

- . The direction vector (the center of view)
- . The camera plane vector (left and right spread)

We get a viewing cone like this:



This cone is what the player sees — everything inside it will be rendered.

. . .



Part 5: Ray Direction Formula

To render the 3D world, we send one ray for each vertical stripe of pixels on the screen.

To calculate each ray's direction:

```
rayDir = direction + cameraPlane * cameraX
```

Where:

```
direction = player direction vector  
cameraX = a value from -1 (left edge) to +1 (right edge)  
cameraPlane = left/right direction vector (FOV)
```

This means:

```
The leftmost ray = dir - plane  
The center ray = dir  
The rightmost ray = dir + plane
```

So the rays sweep across the **FOV** — and that's how the screen gets rendered column by column.

. . .



DDA Algorithm in Ray-casting (Cub3D Style)



Purpose of DDA “Digital Differential Analysis”

You want to render 3D walls based on how far each ray travels before it hits a wall.

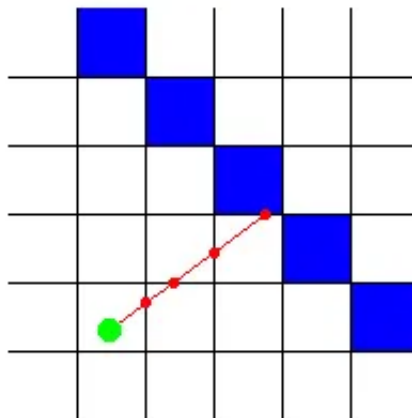
To do that, you must figure out:

“Which square (tile) on the 2D map does my ray hit first?”

This is exactly what DDA helps you do:

Trace a ray step-by-step through a grid (the map).
Check at each step whether you've hit a wall (`map[x][y] == '1'`).
Stop when a wall is hit and record the distance.

In order to render 3D walls correctly, we need to calculate the distance from the player to the point where a ray intersects a wall. To achieve this, we use an algorithm known as Digital Differential Analysis (DDA).



DDA is a fast and efficient method designed to work on square grids. It's often used in computer graphics to trace the path of a line through a grid, such as pixels on a screen. In our case, we use DDA to trace rays through the map grid and determine which cells the ray passes through. The process continues until the ray hits a cell that represents a wall.

The illustration above demonstrates how the algorithm progresses step by step, checking each cell along the ray's path. As soon as a cell marked as a wall is encountered, the ray stops.

Because the grid is made up of uniform squares (each 1 unit apart), we can calculate the total distance the ray traveled by multiplying the number of steps by the constant delta values. These deltas represent how far the ray moves in the X and Y directions between each step.

Step 1: Start ray-casting for the whole screen

Goal:

Cast one ray per vertical screen column to create the 3D scene.

Calculate angular increment per ray:

. The player's field of view (FOV) is the total angular range visible on the screen — often something like 60° (which is $\pi/3$ radians).

Get Abdilah CH's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe

☒ Remember me for faster sign in

. The screen is divided into vertical columns (pixels) — say WIDTH columns.

. To cover the entire FOV evenly, each ray must be separated by a small angle increment called pov (plane of view):

$$\text{pov} = (\text{FOV}) / \text{WIDTH} = (\pi/3) / \text{WIDTH}$$

This means each ray will be cast at a slightly different angle, moving from the left edge of the FOV to the right.

🎯 Set starting ray angle to left edge of FOV:

The player's current viewing direction is `player->angle`, which points straight ahead (center of the screen).

The left edge of the field of view is offset from the center by half the FOV:

```
starting_x = player->angle - (π/6)    // player->angle is center view
```

🔄 Loop through every vertical column on the screen

For each column `i` from 0 to `WIDTH-1`:

Cast the ray:

```
Cast a ray into the map at angle starting_x
Calculate where it hits a wall
Determine how far away the wall is
Draw a vertical slice of the wall at column i, scaled by distance to cr
```

Increment the ray angle:

```
starting_x=starting_x+pov
```

• • •



Step 2: Initialize ray parameters & prepare for grid traversal



Goal

Before we step the ray through the 2D map grid using DDA, we need to:

- . Know where the ray starts (based on player's position),
- . Know how much to move in each step (based on angle),
- . Know which direction to step in (left/right, up/down),
- . Know the initial distance to the first vertical and horizontal grid lines



Convert player's position to grid cell coordinates

Your map is a grid, and each cell is typically 64×64 pixels (common in Cub3D). So to know which cell the player is in:

```
dda.x = (int)(player->y / 64); // Column in the map grid
dda.y = (int)(player->x / 64); // Row in the map grid
```

You divide by 64 to get cell indices instead of pixel positions.



Calculate how far we must go to hit the next vertical and horizontal lines

These values tell us how far (in world units) a ray needs to go **horizontally** or **vertically** to cross into the next grid cell.

```
dda.x_line_length = fabs(1 / cos(ray_angle)); // Distance to next vertical line
dda.y_line_length = fabs(1 / sin(ray_angle)); // Distance to next horizontal line
```

Why use `1 / cos()` and `1 / sin()`?

Think of it like this:

- . When we step in X, how far do we move in Y to keep the ray straight?
- . $1 / \cos$ gives how far the ray moves in world space per X grid cell step.
- . This creates the scaling factor for each step.

? Decide step direction (+1 or -1)

We must determine if the ray moves left/right and up/down, based on the ray angle:

```
if (cos(ray_angle) < 0) x_step = -1; // Left
else x_step = 1;                      // Right

if (sin(ray_angle) < 0) y_step = -1; // Up
else y_step = 1;                    // Down
```

This tells DDA which direction to step in each axis during traversal.

Calculate initial distance to the first vertical/horizontal grid line

We now compute how far the ray must travel from the player's current position to reach the first vertical and horizontal grid lines.

For vertical lines (X steps):

```
if (x_step == -1)
    dda.x_dist = (player->y / 64 - dda.x) * dda.x_line_length;
else
    dda.x_dist = (dda.x + 1 - player->y / 64) * dda.x_line_length;
```

This gives distance from current position to the next vertical grid boundary

in the direction of the ray.

For horizontal lines (Y steps):

```
if (y_step == -1)
    dda.y_dist = (player->x / 64 - dda.y) * dda.y_line_length;
else
    dda.y_dist = (dda.y + 1 - player->x / 64) * dda.y_line_length;
```

Same idea, but for horizontal intersections.

These distances are the starting points for DDA traversal.

. . .

Step 3: Traverse the grid with DDA to find wall hit

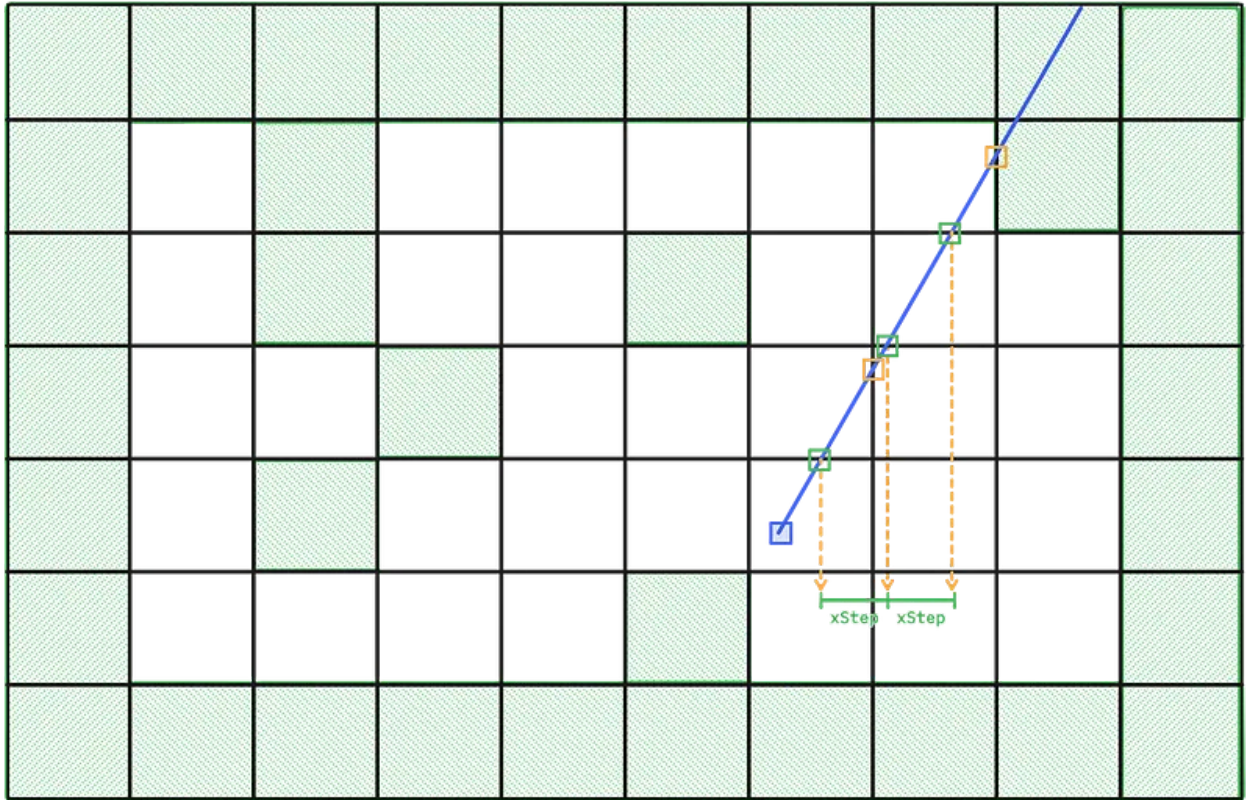
Once you've initialized everything (starting distances, steps, directions), it's time to let the DDA algorithm do its work: walk along the ray, cell by cell, until a wall is found.

 **Loop until a wall is hit**

```
while (!reached_wall) // This loop continues until the ray hits a wall.
```

Decide whether to move in X or Y direction

In each loop iteration, you choose which side of the grid the ray hits next — vertical or horizontal — by comparing the accumulated distances.



◆ **If next X step is closer:**

```
if (dda.x_dist < dda.y_dist) // This means the ray will cross a vertical
```

So :

```
dda.x_dist += dda.x_line_length; // move one X-step ahead
dda.x += dda.x_step;             // move in map cell grid
dda.wall_face = 0;               // vertical wall hit
```

◆ **Else → next Y step is closer:**

```
dda.y_dist += dda.y_line_length; // move one Y-step ahead
dda.y += dda.y_step;             // move in map cell grid
```

```
dda.wall_face = 1; // horizontal wall hit
```

This mimics the **Bresenham/DDA-style line algorithm** and ensures you're checking the **closest intersection** first.

☎ **Check if this grid cell is a wall**

After stepping, check the current grid cell in the map to see if it's a wall:

```
reached_wall = touch(player, dda.y, dda.x);
```

The touch() function typically checks:

```
if (map[dda.y][dda.x] == '1') // wall found
    return true;
else
    return false;
```

If it's a wall, exit the loop — we've found the **first wall the ray hits**.

Calculate the true distance from player to wall

After exiting the loop, we need to calculate how far the ray traveled. But remember: during the last step, we added an extra line length to dda.x_dist or dda.y_dist, so we need to subtract one step:

```
if (dda.wall_face == 0)
    dda.distance = dda.x_dist - dda.x_line_length; // vertical
else
    dda.distance = dda.y_dist - dda.y_line_length; // horizontal
```

Now `dda.distance` contains the **real distance** from the player to the wall.

Fisheye Correction

Why do we need this?

Because the ray angle might differ from the player's center view. If you don't correct for this, walls at the edge of the screen look curved or warped — this is called the fisheye effect.

To fix it, we project the ray distance onto the player's direction vector:

```
dda.distance *= cos(player->angle - ray_angle); // This ensures that th
```

Rendering Wall Slices in 3D with Textures

After performing DDA to detect a wall, we now need to:

 Turn the wall hit into a vertical stripe on screen using the correct texture and height.

Step 1: Choose the Right Texture

Goal:

Figure out which texture (NORTH, SOUTH, EAST, WEST, DOOR) to use for the wall.

How:

When a ray hits a wall, it can hit either:

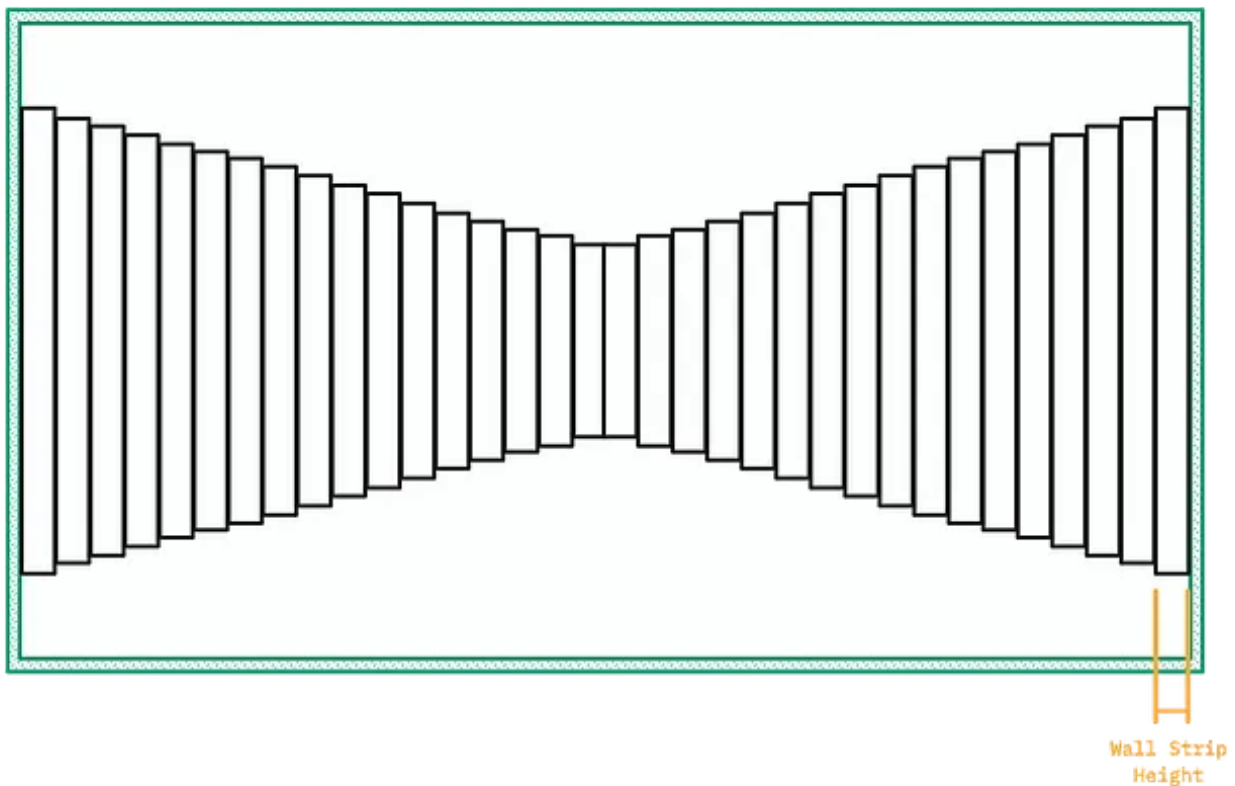
- . A vertical wall (`wall_face == 0`),
- . Or a horizontal wall (`wall_face == 1`).

Depending on the direction of the step (was the ray going right/left or up/down?), we know which face of the wall we're looking at.

🧠 Logic:

```
if (wall_face == 1) // horizontal wall
{
    if (y_step > 0)
        return north_texture;
    else
        return south_texture;
}
else // vertical wall
{
    if (x_step > 0)
        return west_texture;
    else
        return east_texture;
}
```

📌 Step 2: Compute X Coordinate Inside Texture



✅ Goal:

Figure out which column of the texture the ray is hitting.

 Why:

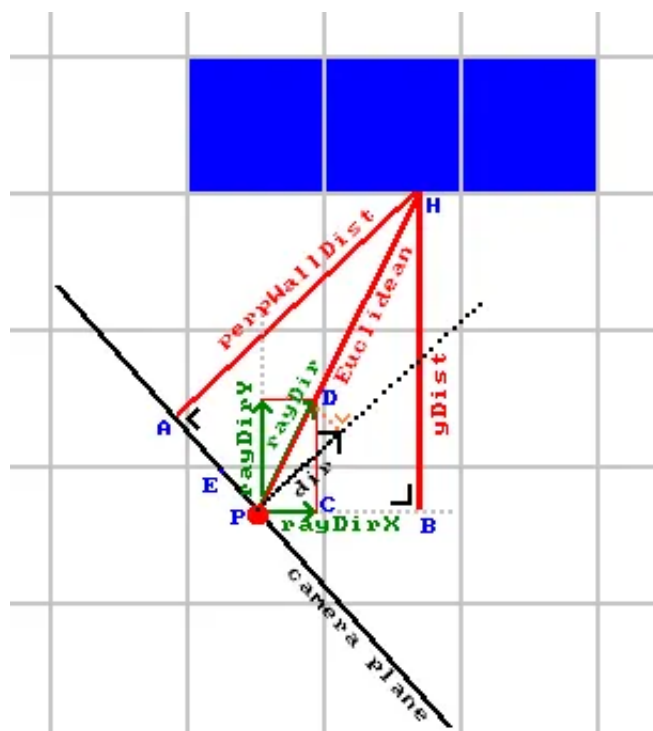
A wall is textured horizontally, like a wallpaper. You need to figure out where exactly the ray hit along the wall (left edge? center? near right?).

1. Undo fisheye correction for texture alignment:

```
true_distance = player->dda.distance / cos(player->angle - ray_angle);
```

2. Find wall hit coordinate in map:

```
if (vertical wall)
    wallx = player->x + true_distance * sin(ray_angle);
else
    wallx = player->y + true_distance * cos(ray_angle);
```



3. Normalize it to 0–1 (fraction of wall):

```
wallx /= 64.0;           // Convert to cell-relative coordinate  
wallx -= floor(wallx);    // Get only the decimal part
```

1. Scale to texture width:

```
texx = wallx * texture->width;
```

2. Flip the texture (mirror) in some cases:

```
if (wall_face == 0 && ray goes right)  
    texx = texture->width - texx - 1;  
if (wall_face == 1 && ray goes up)  
    texx = texture->width - texx - 1;
```

This ensures textures are not mirrored wrongly depending on ray direction.

• • •

Step 3: Draw the Wall Slice

Goal:

Render the wall slice in 3 parts:

- . Ceiling
- . Textured wall
- . Floor

Calculate Wall Height:

```
111 // calculate wall height from the center of the screen
```

Sign up

Sign in

Medium



- Multiply by `WIDTH` as a scaling factor (you can adjust this if it feels too big or too small).

Calculate vertical drawing bounds

```
start_y = (HEIGHT / 2) - (wall_height / 2); // top of the wall
end_y   = start_y + wall_height;           // bottom of the wall
```

This centers the wall vertically on screen.

• • •

A. Draw Ceiling (before wall)

```
while (j < start_y)
    put_pixel(p, i, j++, p->config->ceiling_color);
```



Loop from `start_y` to `end_y`:

```
while (j < end_y)
{
    if (j >= 0 && j < HEIGHT)
    {
        // Compute y coordinate in texture
        texy = (int)((j - start_y) * ((double)tex->height / wall_height));
        // Get color from texture
        p->color = tex->data[texy * tex->width + texX];
        put_pixel(p, i, j, p->color);
        // Draw it
    }
    j++;
}
```

You're mapping vertical pixel `j` to vertical texture row `texy`.

C. Draw Floor (after wall)

```
for (; j < HEIGHT; j++)
    put_pixel(player, i, j, floor_color);
```



Follow

Written by Abdilah CH

14 followers · 1 following

Student at 1337, passionate about coding, problem-solving, and low-level programming. Follow along for insights, challenges, and deep dives into coding!

No responses yet

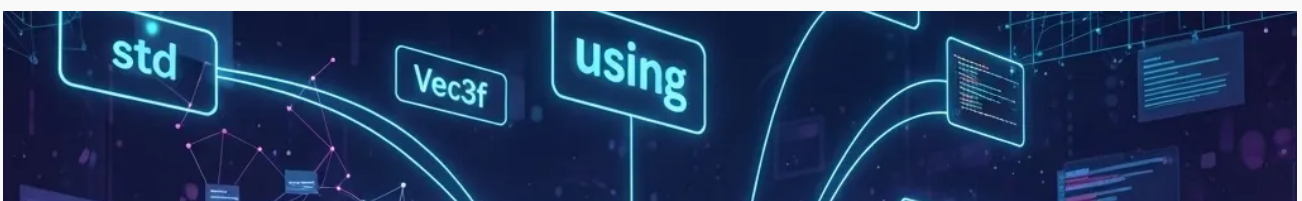


To respond to this story,
get the free Medium app.

Continue in app



More from Abdilah CH



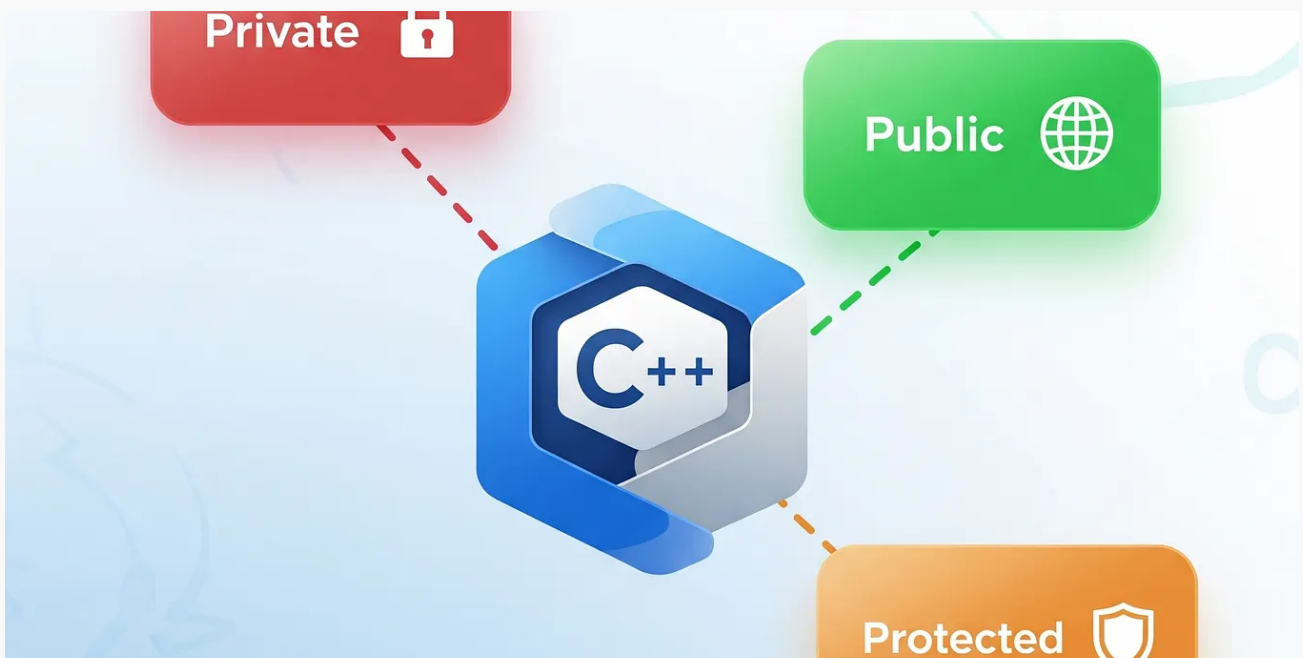


 Abdilah CH

“using” Keyword in C++

The using keyword in C++ is a versatile feature that serves multiple purposes, primarily related to namespace management, type aliasing...

Sep 3, 2025  1



 Abdilah CH

C++ Access Specifiers: Private, Public, and Protected Explained

In C++, access specifiers control the visibility and accessibility of class members (fields, methods, etc.) from different parts of the...

Aug 27, 2025





Abdilah CH

Virtual function table in c++

In C++98, the vtable (virtual function table) is a key mechanism used to implement runtime polymorphism through virtual functions. It...

Oct 6, 2025





Abdilah CH

Exception Handling in C++

1. What Is an Exception?

Dec 18, 2025



See all from Abdilah CH

Recommended from Medium



In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-engineering in any OO language

over engineering many C++ languages.

★ Jan 29 🖱 6.7K 💬 60



AmirHossein Aghajari

Liquid Glass: iOS Effect Explanation

Apple's Liquid Glass effect with shaders: math, distortion & chromatic aberration

Nov 24, 2025 🖱 239 💬 1





In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model—no training required...



Oct 20, 2025



25K



685



In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.



Dec 28, 2025



11.6K



417





In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big Tech AI Subscriptions Forever |...

Something strange happened in early 2026. Apple stores started running low on Mac Minis. Tech forums exploded with setup guides. Developers...



Feb 15



5.6K



96



In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy Your Brain

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)



Jan 14



40K



750



See more recommendations